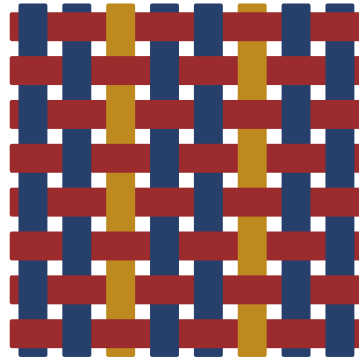




Information Theory

Lecture 2 · from codewords to typical sequences

Oxford B8.4 · 2024--25

fanzuituanhuo

August 18, 2026

dyed in indigo, madder & weld; ruled in iron-gall. woven on LOOM.

THE ONE IDEA, AND THE MAP

□□□□

A source code replaces each source symbol by a finite string. Its quality has two sides: **efficiency**, measured by expected length, and **decodability**, tested after codewords are concatenated. Kraft-McMillan controls which lengths fit; the AEP then shows why entropy is the natural scale for coding long i.i.d. blocks.

The map so far

stop	object	question
1	$C : \mathcal{X} \rightarrow \mathcal{D}^*$	Which finite string represents each source symbol?
2	$L(C) = \mathbb{E}[C(X)]$	How many code symbols are used on average?
3	$C^* : \mathcal{X}^* \rightarrow \mathcal{D}^*$	Can concatenated codewords still be decoded uniquely?
4	$\sum_x D^{-\ell_x} \leq 1$	Which length lists can belong to a decodable code?
5	$-n^{-1} \log p(X^n) \rightarrow H(X)$	Where does the probability mass of a long block concentrate?
6	$H_D(X) \leq L(C) < H_D(X) + 1$	How short can a uniquely decodable code be?
7	$n^{-1} \mathbb{E}[C(X^n)] \rightarrow H(X)$	What does coding whole blocks buy us?
8	$-n^{-1} \log p(X^n) \rightarrow \bar{H}$	What survives when the source is not i.i.d.?
9	$H_D(X) + D_D(p\ q)$	What does an explicit code cost, and what if it is built for the wrong law?

Conventions

The source alphabet $\mathcal{X}$ is finite, and $\mathcal{D}^*$ is the set of all finite-length strings over a D -ary code alphabet $\mathcal{D}$. Most examples use $\mathcal{D} = \{0, 1\}$.

CONTENTS

The one idea, and the map	1
Lecture 2 · Source coding	3
1 Codes, length, and decodability	3
2 Kraft–McMillan: when lengths fit	6
3 AEP and typical sequences	8
4 Optimal codes and the source coding theorem	12
5 Block codes and Shannon’s source coding theorem	15
6 Two explicit constructions: Shannon and Fano	18

LECTURE 2 · SOURCE CODING

Assign strings to symbols, average their lengths under the source law, and then ask the decisive question: does concatenation preserve enough structure to undo the encoding? For long i.i.d. blocks, identify the typical sequences that carry almost all probability and reveal entropy as the exponential counting rate.

■ 1 CODES, LENGTH, AND DECODABILITY

□□□□ A codeword names one symbol; an extended code must preserve a whole sequence.

Coding begins with a map from source symbols to finite strings. Short strings save space, but distinct symbols and distinct source sequences must remain distinguishable. This is why expected length and decodability have to be studied together.

From a source symbol to a codeword

■ **Definition 1.1** (source code). Let X be a random variable with finite alphabet $\mathcal{X}$. A D -ary source code is a map

$$C : \mathcal{X} \rightarrow \mathcal{D}^*,$$

where $\mathcal{D}^*$ is the set of finite-length strings over a D -ary alphabet $\mathcal{D}$. The string $C(x)$ is the **codeword** of x , and $|C(x)|$ is its length.

For instance, if $\mathcal{X} = \{\text{red}, \text{blue}\}$ and $\mathcal{D} = \{0, 1\}$, one may set $C(\text{red}) = 00$ and $C(\text{blue}) = 11$.

■ **Definition 1.2** (expected length). If X has probability mass function p , the expected length of C is

$$L(C) = \mathbb{E}[|C(X)|] = \sum_{x \in \mathcal{X}} p(x) |C(x)|.$$

■ **Example 1.3** (an entropy-matched binary code). Let $\mathcal{X} = \{1, 2, 3, 4\}$, with probabilities and codewords

x	1	2	3	4
$p(x)$	$\frac{1}{2}$	$\frac{1}{4}$	$\frac{1}{8}$	$\frac{1}{8}$
$C(x)$	0	10	110	111

The codeword lengths are 1, 2, 3, 3, so

$$L(C) = \frac{1}{2} \cdot 1 + \frac{1}{4} \cdot 2 + \frac{1}{8} \cdot 3 + \frac{1}{8} \cdot 3 = \underline{\frac{7}{4}} \text{ bits.}$$

Since $|C(x)| = -\log_2 p(x)$ for every symbol here,

$$H(X) = \sum_x p(x) \log_2 \frac{1}{p(x)} = \underline{\frac{7}{4}} \text{ bits} = L(C).$$

Your turn. Why does the equality $L(C) = H(X)$ require no averaging trick in this example? The equality holds term by term: each length satisfies $|C(x)| = -\log_2 p(x)$. Multiplying by $p(x)$ and summing therefore gives $L(C) = H(X)$.

Three levels of decodability

Definition 1.4 (*nonsingular code*). A code C is **nonsingular** if different source symbols have different codewords:

$$x \neq x' \implies C(x) \neq C(x').$$

Equivalently, C is injective.

Nonsingularity only protects one symbol at a time. To encode a sequence, extend C by concatenation:

$$C^* : \mathcal{X}^* \longrightarrow \mathcal{D}^*, \quad C^*(x_1 x_2 \cdots x_n) = C(x_1) C(x_2) \cdots C(x_n).$$

Definition 1.5 (*uniquely decodable code*). A code is **uniquely decodable** if its extension C^* is nonsingular. Thus every encoded string has at most one source sequence that produced it.

Definition 1.6 (*prefix code*). A code is a **prefix code**, or instantaneous code, if no codeword is a prefix of any other codeword.

Every prefix code is uniquely decodable, and every uniquely decodable code is nonsingular:

$$\{\text{prefix codes}\} \subset \{\text{uniquely decodable codes}\} \subset \{\text{nonsingular codes}\}.$$

The converses need not hold.

Example 1.7 (*uniquely decodable but not prefix*). For $\mathcal{X} = \{A, B, C, D\}$, consider

$$C(A) = 0, \quad C(B) = 01, \quad C(C) = 011, \quad C(D) = 111.$$

This is not a prefix code because 0 is a prefix of both 01 and 011. Nevertheless, it is uniquely decodable. Removing an initial 0 from a longer codeword leaves only 1 or 11; comparing these suffixes with 111 swaps $1 \leftrightarrow 11$ and never produces the empty suffix. Hence no two source sequences have the same concatenation.

Your turn. The code $\{0, 01, 10\}$ is nonsingular but not uniquely decodable. Exhibit two different source sequences with the same encoded string. *AC and BA both encode as 010.*

? Why does nonsingularity not guarantee decodability after concatenation?

■ 2 KRAFT--MCMILLAN: WHEN LENGTHS FIT

□□□□ A decodable code spends a finite amount of room in the code tree.

The Kraft sum turns a list of codeword lengths into a capacity test. Unique decodability forces the sum to be at most one; conversely, every integer length list that passes this test can be packed into a prefix code.

Trigger. Use this test before trying to draw codewords: impossible length lists fail immediately.

The two directions

direction	meaning
necessity	uniquely decodable $\implies \sum_x D^{-\ell_x} \leq 1$
sufficiency	integer lengths with $\sum_x D^{-\ell_x} \leq 1 \implies$ a prefix code with exactly those lengths

Theorem 2.1 (*Kraft–McMillan inequality*). Let $C : \mathcal{X} \rightarrow \mathcal{D}^*$ be a D -ary uniquely decodable code, and write $\ell_x = |C(x)|$. Then

$$\sum_{x \in \mathcal{X}} D^{-\ell_x} \leq 1.$$

Conversely, if positive integers $(\ell_x)_{x \in \mathcal{X}}$ satisfy this inequality, then there exists a D -ary prefix code C such that $|C(x)| = \ell_x$ for every $x \in \mathcal{X}$.

Necessity: amplify by concatenating n symbols

Necessary direction. Set

$$K = \sum_{x \in \mathcal{X}} D^{-\ell_x}, \quad \ell_{\min} = \min_x \ell_x, \quad \ell_{\max} = \max_x \ell_x.$$

For a source string $x^n = (x_1, \dots, x_n)$, its encoded length is $|C^*(x^n)| = \sum_{i=1}^n \ell_{x_i}$. Expanding the n th power gives

$$K^n = \sum_{x^n \in \mathcal{X}^n} D^{-|C^*(x^n)|} = \sum_{k=n\ell_{\min}}^{n\ell_{\max}} A_k D^{-k},$$

where A_k is the number of source strings in $\mathcal{X}^n$ whose encoded output has length k .

Unique decodability makes these A_k output strings distinct. There are only D^k strings of length k over $\mathcal{D}$, so $A_k \leq D^k$. Therefore

$$K^n \leq \sum_{k=n\ell_{\min}}^{n\ell_{\max}} 1 = n(\ell_{\max} - \ell_{\min}) + 1.$$

Taking n th roots and then letting $n \rightarrow \infty$ yields $K \leq 1$. ■

Your turn. Where exactly did unique decodability enter the proof? It was used to turn source-string counting into output-string counting: distinct source strings must give distinct encoded strings, so among the D^k possible strings of length k we can have at most $A_k \leq D^k$.

Sufficiency: pack adjacent D -ary intervals

Sufficient direction. Relabel $\mathcal{X} = \{1, \dots, M\}$ so that $\ell_1 \leq \ell_2 \leq \dots \leq \ell_M$, and define

$$r_m = \sum_{i=1}^{m-1} D^{-\ell_i}, \quad 1 \leq m \leq M.$$

The Kraft inequality gives $0 \leq r_m < 1$. Since $\ell_i \leq \ell_m$ for $i < m$, $D^{\ell_m} r_m$ is an integer, so r_m has a terminating base- D expansion with at most ℓ_m digits. Let $C(m)$ be its first ℓ_m digits, padding with zeros when necessary.

To see that the resulting code is prefix-free, take $m < n$. By construction,

$$r_n - r_m = \sum_{i=m}^{n-1} D^{-\ell_i} \geq D^{-\ell_m}.$$

But every base- D fraction whose first ℓ_m digits equal $C(m)$ lies in the half-open interval $[r_m, r_m + D^{-\ell_m})$. Thus r_n cannot begin with $C(m)$, so $C(m)$ is not a prefix of $C(n)$. Hence C is a prefix code with the prescribed lengths. ■

Example 2.2 (a tight binary packing). For lengths $(1, 2, 3, 3)$,

$$2^{-1} + 2^{-2} + 2^{-3} + 2^{-3} = 1.$$

The construction uses left endpoints $0, \frac{1}{2}, \frac{3}{4}, \frac{7}{8}$, whose binary expansions give the prefix code $0, 10, 110, 111$.

Your turn. Could a binary uniquely decodable code have lengths $(1, 2, 2)$? Compute the Kraft sum and state the verdict.

$$2^{-1} + 2^{-2} + 2^{-2} = \underline{1}.$$

Yes. The Kraft sum is 1, so the converse constructs a prefix code; one choice is $0, 10, 11$.

? Why does the n th-root limit force the Kraft sum to be at most one?

3 AEP AND TYPICAL SEQUENCES

□□□□ A long random block behaves as though its likely outcomes were nearly equiprobable.

The asymptotic equipartition property turns entropy into a geometric picture: almost all probability lies on roughly $2^{nH(X)}$ length- n sequences, and each of those sequences has probability roughly $2^{-nH(X)}$.

Trigger. Use typical sets when a one-symbol distribution must be converted into a statement about long blocks.

A Bernoulli preview

Example 3.1 (*counting zeros and ones*). Let $X \sim \text{Bernoulli}(p)$ and let $X_1, \dots, X_n$ be i.i.d. copies. If a realization $x^n = (x_1, \dots, x_n)$ contains $N_1(x^n)$ ones, then

$$p_{X^n}(x^n) = p^{N_1(x^n)}(1-p)^{n-N_1(x^n)}.$$

The weak law of large numbers says $N_1(X^n)/n \rightarrow p$ in probability. Thus, for most long realizations,

$$\begin{aligned} -\frac{1}{n} \log p_{X^n}(X^n) &\approx -p \log p - (1-p) \log(1-p) \\ &= \underline{H(X)}. \end{aligned}$$

Equivalently, a typical realization has probability about $2^{-nH(X)}$.

Three modes of random-variable convergence

Definition 3.2 (*convergence in probability*). Let $Y_1, Y_2, \dots$ and Y be real-valued random variables on the same probability space. We say that Y_n **converges in probability** to Y , and write $Y_n \xrightarrow{P} Y$, if for every $\epsilon > 0$,

$$\mathbb{P}(|Y_n - Y| > \epsilon) \rightarrow \underline{0} \quad \text{as } n \rightarrow \infty.$$

Thus convergence in probability controls the probability of a fixed-size error; it does not assert that every sample path eventually stays close.

Definition 3.3 (*mean-square convergence*). Suppose $Y_1, Y_2, \dots$ and Y are square-integrable. We say that Y_n **converges in mean square**, or in L^2 , to Y if

$$\mathbb{E}[|Y_n - Y|^2] \rightarrow 0.$$

The notation is $Y_n \xrightarrow{L^2} Y$. This asks the average squared error, not merely the probability of a large error, to vanish.

Definition 3.4 (*almost-sure convergence*). We say that Y_n **converges almost surely**, or with probability one, to Y if

$$\mathbb{P}\left(\left\{\omega : \lim_{n \rightarrow \infty} Y_n(\omega) = Y(\omega)\right\}\right) = 1.$$

The notation is $Y_n \xrightarrow{\text{a.s.}} Y$. Thus, outside a single probability-zero exceptional set, the numerical sequence $Y_n(\omega)$ converges to $Y(\omega)$ along every remaining sample path.

The implications needed most often are

$$Y_n \xrightarrow{L^2} Y \implies Y_n \xrightarrow{\text{P}} Y, \quad Y_n \xrightarrow{\text{a.s.}} Y \implies Y_n \xrightarrow{\text{P}} Y.$$

The first follows from $\mathbb{P}(|Y_n - Y| > \epsilon) \leq \mathbb{E}[|Y_n - Y|^2]/\epsilon^2$. In general, mean-square and almost-sure convergence do not imply one another without extra assumptions. (For the AEP's i.i.d. variables both do hold: finite variance gives L^2 convergence, and the strong law of large numbers gives almost-sure convergence; convergence in probability is the weakest of the three — and all that weak typicality needs.)

The weak law of large numbers

Theorem 3.5 (*weak law of large numbers*). Let $Y_1, Y_2, \dots$ be i.i.d. with finite mean $\mu = \mathbb{E}[Y_1]$ and finite variance. Then

$$\frac{1}{n} \sum_{i=1}^n Y_i \xrightarrow{\text{P}} \mu.$$

In the AEP, $Y_n = -n^{-1} \log p_{X^n}(X_1, \dots, X_n)$ and the limit $Y = H(X)$ is constant; the theorem asserts convergence in probability.

Theorem 3.6 (*weak asymptotic equipartition property*). Let X be a discrete random variable on a finite alphabet, and let $X_1, X_2, \dots$ be i.i.d. copies of X . Then

$$-\frac{1}{n} \log p_{X^n}(X_1, \dots, X_n) \longrightarrow H(X) \quad \text{in probability.}$$

Proof. On the support of X , define $Z_i = -\log p_X(X_i)$. Independence gives

$$-\log p_{X^n}(X_1, \dots, X_n) = \sum_{i=1}^n -\log p_X(X_i) = \sum_{i=1}^n Z_i.$$

The variables Z_i are i.i.d. and

$$\mathbb{E}[Z_i] = \sum_x p_X(x) \log \frac{1}{p_X(x)} = H(X).$$

The weak law of large numbers applied to $n^{-1} \sum_i Z_i$ proves the claim. ■

Your turn. What is the one-symbol random variable to which the law of large numbers is applied in the AEP proof? Apply the law of large numbers to the surprise $Z = -\log p_X(X)$. Its expectation is exactly $H(X)$.

Weak typicality: make "about" precise

Definition 3.7 (*weakly typical set*). For $\epsilon > 0$, the length- n weakly typical set is

$$T_{n,\epsilon} = \left\{ x^n \in \mathcal{X}^n : \left| -\frac{1}{n} \log p_{X^n}(x^n) - H(X) \right| \leq \epsilon \right\}.$$

Its members are the ϵ -typical sequences of length n .

Theorem 3.8 (*typical-set properties*). For every $\epsilon > 0$, all sufficiently large n satisfy:

1. every $x^n \in T_{n,\epsilon}$ obeys

$$2^{-n(H(X)+\epsilon)} \leq p_{X^n}(x^n) \leq 2^{-n(H(X)-\epsilon)};$$

2. $\mathbb{P}(X^n \in T_{n,\epsilon}) > 1 - \epsilon$;

3. the cardinality is bounded by

$$(1 - \epsilon)2^{n(H(X)-\epsilon)} < |T_{n,\epsilon}| \leq 2^{n(H(X)+\epsilon)}.$$

The first property follows from the definition for every n ; only the latter two require n to be large.

Proof. The first claim is obtained by exponentiating the defining inequality. The AEP gives

$$\mathbb{P}(X^n \notin T_{n,\epsilon}) = \mathbb{P}\left(\left| -\frac{1}{n} \log p_{X^n}(X^n) - H(X) \right| > \epsilon\right) \rightarrow 0,$$

which proves the second claim.

For the upper cardinality bound, use the lower probability bound on each typical sequence:

$$1 \geq \sum_{x^n \in T_{n,\epsilon}} p_{X^n}(x^n) \geq |T_{n,\epsilon}| 2^{-n(H(X)+\epsilon)}.$$

For the lower cardinality bound, use the upper probability bound together with the mass estimate:

$$1 - \epsilon < \sum_{x^n \in T_{n,\epsilon}} p_{X^n}(x^n) \leq |T_{n,\epsilon}| 2^{-n(H(X)-\epsilon)}.$$

Rearranging gives the stated bounds. ■

Your turn. Complete the AEP slogan by combining the probability and cardinality bounds: there are about $\frac{2^{nH(X)}}{2^{-nH(X)}}$ typical sequences, each with probability about $2^{-nH(X)}$.

The smallest set carrying almost all mass

Definition 3.9 (*minimum high-probability set*). For $0 < \delta < 1$, let $S_m^\delta \subseteq \mathcal{X}^m$ be a set of minimum cardinality subject to

$$\mathbb{P}(X^m \in S_m^\delta) \geq 1 - \delta.$$

This is the “strong” viewpoint in the handwritten notes: keep only the most probable sequences needed to capture the prescribed mass.

Proposition 3.10 (*exponential equivalence of typical sets*). Let (δ_j) be any positive sequence with $\delta_j \rightarrow 0$. Then

$$\lim_{j \rightarrow \infty} \limsup_{m \rightarrow \infty} \left| \frac{1}{m} \log \frac{|S_m^{\delta_j}|}{|T_{m,\delta_j}|} \right| = 0.$$

Thus the minimum high-probability set and the weakly typical set may differ in their individual members, but not in their exponential size as the tolerance shrinks.

Proof. For each fixed $0 < \delta < 1$, the AEP and the typical-set bounds imply

$$\lim_{m \rightarrow \infty} \frac{1}{m} \log |S_m^\delta| = H(X), \quad H(X) - \delta \leq \liminf_m \frac{1}{m} \log |T_{m,\delta}| \leq \limsup_m \frac{1}{m} \log |T_{m,\delta}| \leq H(X) + \delta.$$

For the first limit, an upper bound comes from choosing a typical set; a lower bound follows by intersecting any set of mass at least $1 - \delta$ with an η -typical set and then letting $\eta \downarrow 0$. Hence the normalized logarithm of the ratio has absolute limsup at most δ . Substituting $\delta = \delta_j$ and letting $j \rightarrow \infty$ proves the result. ■

? How do “probability per typical sequence” and “number of typical sequences” multiply back to one?

4 OPTIMAL CODES AND THE SOURCE CODING THEOREM

□□□□ Kraft–McMillan turns “find the best code” into a constrained minimization whose answer is $-\log_D p(x)$.

Entropy is the universal lower bound on expected length; a Shannon code comes within one D -ary symbol of it.

Trigger. Use this section when a code’s expected length must be compared with an information-theoretic floor.

What “optimal” means

Definition 4.1 (*optimal code*). A symbol code $C : \mathcal{X} \rightarrow \mathcal{D}^*$ is **optimal** for a random variable X with pmf p if it minimizes $\mathbb{E}[|C(X)|]$ among all uniquely decodable codes $C' : \mathcal{X} \rightarrow \mathcal{D}^*$.

The competition is over uniquely decodable codes, not over prefix codes; by Kraft–McMillan the two classes have the same achievable length lists, so nothing is lost by looking for the optimum inside the prefix codes.

The optimization problem behind the definition

Write $\ell_x = |C(x)|$. Kraft–McMillan says a length list is realizable by a uniquely decodable code exactly when it passes the Kraft test, so optimality is the constrained problem

$$\text{minimize } \sum_{x \in \mathcal{X}} p(x) \ell_x \quad \text{subject to } \sum_{x: p(x) > 0} D^{-\ell_x} \leq 1, \quad \ell_x \in \mathbb{N}.$$

The relaxed solution by Lagrange multipliers

Ignore the integer constraint, treat the ℓ_x as real, and impose the Kraft constraint with equality. The Lagrangian is

$$J(\ell, \lambda) = \sum_x p(x) \ell_x - \lambda \left(\sum_x D^{-\ell_x} - 1 \right).$$

Setting $\partial J / \partial \ell_x = 0$ gives $p(x) + \lambda D^{-\ell_x} \ln D = 0$, that is $D^{-\ell_x} = -p(x) / (\lambda \ln D)$. Summing over x and using the constraint $\sum_x D^{-\ell_x} = 1$ forces $-\lambda \ln D = 1$, hence

$$D^{-\ell_x} = p(x), \quad \ell_x = \underline{-\log_D p(x)}.$$

The objective is convex in ℓ and the constraint set is convex, so this stationary point is a minimum. Its value is the average length

$$\mathbb{E}[|C(X)|] = \sum_x -p(x) \log_D p(x) = H_D(X).$$

Here $H_D(X) = \sum_x p(x) \log_D \frac{1}{p(x)} = H(X) / \log_2 D$ is entropy measured in D -ary units. For $D = 2$ it is the usual entropy in bits.

The floor is entropy

Theorem 4.2 (source coding theorem, one symbol at a time). Let X take finitely many values in $\mathcal{X}$ and let C be a uniquely decodable D -ary source code. Then

$$H_D(X) \leq \mathbb{E}[|C(X)|],$$

with equality if and only if $-\log_D p(x) \in \mathbb{N}$ for every x with $p(x) > 0$.

Proof. Set $\ell_x = |C(x)|$ and define the auxiliary pmf

$$q(x) = \frac{D^{-\ell_x}}{\sum_{x'} D^{-\ell_{x'}}}, \quad c = \sum_{x'} D^{-\ell_{x'}}.$$

Then

$$\begin{aligned} \mathbb{E}[|C(X)|] - H_D(X) &= \sum_x p(x)\ell_x + \sum_x p(x) \log_D p(x) \\ &= -\sum_x p(x) \log_D (D^{-\ell_x}) + \sum_x p(x) \log_D p(x) \\ &= -\sum_x p(x) \log_D (q(x)c) + \sum_x p(x) \log_D p(x) \\ &= -\log_D c + \sum_x p(x) \log_D \frac{p(x)}{q(x)} \\ &= -\log_D c + D_D(p \| q) \geq 0. \end{aligned}$$

The last step uses two facts: $c \leq 1$ by Kraft–McMillan, so $-\log_D c \geq 0$; and relative entropy is nonnegative. Equality forces both terms to vanish, i.e. $c = 1$ and $p = q$, which is exactly $D^{-\ell_x} = p(x)$, that is $\ell_x = -\log_D p(x) \in \mathbb{N}$. ■

Your turn. The proof splits the gap $\mathbb{E}[|C(X)|] - H_D(X)$ into two nonnegative pieces. Name each piece and say which property makes it nonnegative. The slack term $-\log_D c$ with $c = \sum_x D^{-\ell_x}$ is nonnegative because Kraft–McMillan gives $c \leq 1$; the divergence $D_D(p \| q)$ is nonnegative by Gibbs' inequality. The gap closes only when the code fills the code tree exactly ($c = 1$) and matches the source ($q = p$).

Rounding up costs at most one symbol

Proposition 4.3 (Shannon code). Let X be as above. There exists a prefix code $\tilde{C}$ with

$$H_D(X) \leq \mathbb{E}[|\tilde{C}(X)|] < H_D(X) + 1,$$

and hence an optimal code obeys the same bounds.

Proof. Set $\ell_x = \lceil -\log_D p(x) \rceil$. Since $\ell_x \geq -\log_D p(x)$,

$$\sum_x D^{-\ell_x} \leq \sum_x D^{\log_D p(x)} = \sum_x p(x) = 1,$$

so by Kraft–McMillan there exists a prefix code with exactly these lengths. By definition of the ceiling,

$$-\log_D p(x) \leq \ell_x < -\log_D p(x) + 1.$$

Averaging over x with weights $p(x)$ gives the stated bounds. An optimal code cannot be longer than this one, and by Theorem 4.2 it cannot be shorter than $H_D(X)$. ■

Your turn. Why is the Kraft test still passed after replacing $-\log_D p(x)$ by its ceiling? Increasing a length decreases D^{-l_x} , so the Kraft sum can only decrease: it stays at most $\sum_x p(x) = 1$.

Your turn. Where does the “+1” in the Shannon-code bound come from, and why is it harmless once we encode long blocks? It is the cost of forcing integer lengths: each symbol loses less than one code symbol to rounding. Coding blocks of n symbols spreads that single loss over n source symbols, leaving an excess of at most $1/n$ per symbol.

? Which two inequalities in the proof become equalities exactly when $L(C) = H_D(X)$?

■ 5 BLOCK CODES AND SHANNON'S SOURCE CODING THEOREM

□□□□ Encode n symbols at once, and the rounding loss is shared by all of them.

Applying a Shannon code to blocks of n i.i.d. symbols keeps the entropy floor but divides the $+1$ rounding penalty by n . Letting n grow shows that the entropy rate is achievable to any accuracy: this is Shannon's 1948 source coding theorem.

Trigger. Use blocks whenever a per-symbol bound is off by an additive constant that does not shrink on its own.

The basic idea

Consider the $\mathcal{X}^n$ -valued random variable $(X_1, \dots, X_n)$, where $X_1, \dots, X_n$ are i.i.d. copies of X . Independence makes entropy additive,

$$H(X_1, \dots, X_n) = \underline{nH(X)},$$

so Proposition 4.3, applied to the block variable rather than to a single symbol, gives

$$nH(X) \leq \mathbb{E}[|C(X_1, \dots, X_n)|] < nH(X) + 1$$

whenever C is an optimal binary code for the block. Dividing by n turns this into a per-symbol statement:

$$H(X) \leq \frac{1}{n} \mathbb{E}[|C(X_1, \dots, X_n)|] < H(X) + \underline{\frac{1}{n}}.$$

The excess is now under our control: choose n large and it disappears.

Shannon's theorem

Theorem 5.1 (source coding theorem, Shannon 1948). Let $X_1, X_2, \dots$ be an i.i.d. sequence of discrete random variables with state space $\mathcal{X}$. For every $\epsilon > 0$ there exist $n \in \mathbb{N}$ and a map

$$C : \mathcal{X}^n \longrightarrow \{0, 1\}^*$$

such that:

1. the induced map $C^* : \bigcup_{k=0}^{\infty} \mathcal{X}^{nk} \rightarrow \{0, 1\}^*$ given by

$$(x_1, \dots, x_{nk}) \mapsto C(x_1, \dots, x_n) C(x_{n+1}, \dots, x_{2n}) \cdots C(x_{(k-1)n+1}, \dots, x_{nk})$$

is injective;

2. $\frac{1}{n} \mathbb{E}[|C(X_1, \dots, X_n)|] \leq H(X) + \epsilon.$

Proof. Choose n with $1/n \leq \epsilon$, and regard $Y = (X_1, \dots, X_n)$ as a single random variable on the finite alphabet $\mathcal{X}^n$. By Proposition 4.3 with $D = 2$ there is a binary prefix code $C : \mathcal{X}^n \rightarrow \{0, 1\}^*$ with

$$H(Y) \leq \mathbb{E}[|C(Y)|] < H(Y) + 1.$$

Since the X_i are independent, $H(Y) = nH(X)$, so

$$\frac{1}{n} \mathbb{E}[|C(Y)|] < H(X) + \frac{1}{n} \leq H(X) + \epsilon,$$

which is the second claim. For the first, note that C is a prefix code, hence uniquely decodable, so its extension to concatenated blocks is injective: a decoder reads the encoded string from the left, recovers one block codeword at a time, and thereby recovers $(x_1, \dots, x_{nk})$. Thus C^* is injective. ■

Your turn. Which property of C makes the induced map C^* injective, and why is nonsingularity of C alone not enough? **Prefix-freeness.** It lets the decoder parse the stream block by block without lookahead. A nonsingular code only distinguishes single blocks; after concatenation two different block sequences can produce the same string, as in the code $\{0, 01, 10\}$.

Two routes to the same rate

route	how entropy appears
Shannon code on blocks	lengths $[-\log_2 p(x^n)]$ average to $nH(X) + O(1)$
typical sets (§3)	index the $\approx 2^{nH(X)}$ typical sequences with $\approx nH(X)$ bits, and spend a flag bit on the rest

Your turn. Using the AEP route, roughly how many bits per symbol are needed to index the typical set, and what happens to the atypical sequences? **Indexing** $|T_{n,\epsilon}| \leq 2^{n(H(X)+\epsilon)}$ sequences needs about $H(X)+\epsilon$ bits per symbol. The atypical sequences carry probability less than ϵ , so even a wasteful $\log_2 |\mathcal{X}|$ bits per symbol for them contributes a vanishing amount to the average.

Beyond i.i.d.: the entropy rate

Everything above used independence only once, to write $H(X_1, \dots, X_n) = nH(X)$. For a dependent source that quantity is replaced by a limit.

Definition 5.2 (entropy rate). For a stationary sequence $X_1, X_2, \dots$ the **entropy rate** is

$$\bar{H} = \lim_{n \rightarrow \infty} \frac{1}{n} H(X_1, \dots, X_n),$$

which exists because $H(X_1, \dots, X_n)$ is subadditive in n .

Theorem 5.3 (Shannon–McMillan–Breiman). Let $X_1, X_2, \dots$ be an ergodic stationary sequence on a finite state space $\mathcal{X}$. Then

$$-\frac{1}{n} \log p_{X^n}(X_1, \dots, X_n) \longrightarrow \bar{H} \quad \text{in probability, as } n \rightarrow \infty,$$

where $\bar{H} = \lim_{n \rightarrow \infty} \frac{1}{n} H(X_1, \dots, X_n)$.

This is the AEP with independence weakened to *ergodic stationarity* and $H(X)$ replaced by $\bar{H}$; the statement even holds almost surely. Once it is available, typical sets, block coding and Theorem 5.1 all go through verbatim with the entropy rate in place of the single-symbol entropy.

Your turn. For an i.i.d. sequence, what does $\bar{H}$ reduce to, and which hypothesis of Theorem 5.3 replaces independence? Additivity gives $H(X_1, \dots, X_n) = nH(X_1)$, so $\bar{H} = H(X_1)$ and the theorem becomes the AEP. Independence is replaced by the assumption that the sequence is stationary and ergodic, which is what lets the time average of the surprise converge to its ensemble mean.

? Why does blocking remove the +1 penalty but never the entropy floor?

6 TWO EXPLICIT CONSTRUCTIONS: SHANNON AND FANO

□□□□ Kraft only promised that a code exists; here are two recipes that build one.

Proposition 4.3 produced the lengths $\lceil -\log_D p(x) \rceil$ but not the codewords. Shannon's recipe reads them off the cumulative distribution function; Fano's recipe splits the alphabet into balanced halves. Shannon's version also reveals the price of coding with the *wrong* distribution: exactly a relative entropy.

Trigger. Use this section when you need an actual codeword, or when the code was designed for a distribution other than the true one.

Shannon's code

The idea is to give x a codeword of length $\ell_x = \lceil \log_D \frac{1}{p(x)} \rceil$, and to choose *which* string of that length by cutting the D -ary expansion of the cumulative probability.

Definition 6.1 (*Shannon code*). Let $\mathcal{X} = \{x_1, \dots, x_m\}$ with pmf p and let $|\mathcal{D}| = D$.

1. Order the probabilities in decreasing order, $p_1 \geq p_2 \geq \dots \geq p_m$.
2. Set the cumulative probability $\alpha_k = \sum_{i=1}^{k-1} p_i$, and define $C(x_k)$ to be the first

$$\ell_k = \lceil \log_D(1/p_k) \rceil$$

digits in the D -ary expansion of α_k .

The truncation error satisfies $0 \leq \alpha_k - 0.C(x_k) < D^{-\ell_k} \leq p_k$, and $\alpha_{k+1} - \alpha_k = p_k$, so distinct codewords describe disjoint subintervals of $[0, 1)$. Hence the code is a prefix code with exactly the lengths used in Proposition 4.3.

Your turn. Why does the construction need the probabilities sorted, and where would the argument fail without it? *Sorting makes $\ell_1 \leq \ell_2 \leq \dots$, so the interval attached to a later symbol is shorter than the truncation resolution of every earlier one; no codeword can then be a prefix of a previous codeword. Without sorting a long, low-probability interval could contain the truncation point of a later symbol and prefix-freeness could fail.*

Coding with the wrong distribution

Suppose we build the Shannon code from a pmf q while the source really follows p . The excess cost is exactly the relative entropy $D_D(p \| q)$.

Proposition 6.2 (*mismatched Shannon code*). Let p and q be pmfs on $\mathcal{X}$, let $X \sim p$, and let $\mathcal{D}$ be a finite code alphabet with $|\mathcal{D}| = D$. If C is the Shannon code built from the distribution q , then

$$H_D(X) + D_D(p \| q) \leq \mathbb{E}[|C(X)|] < H_D(X) + D_D(p \| q) + 1.$$

Proof. By construction $|C(x)| = \lceil \log_D \frac{1}{q(x)} \rceil$, so

$$\begin{aligned} \mathbb{E}[|C(X)|] &= \sum_x p(x) \left\lceil \log_D \frac{1}{q(x)} \right\rceil \\ &< \sum_x p(x) \left(\log_D \frac{1}{q(x)} + 1 \right) \\ &= \sum_x p(x) \left(\log_D \frac{p(x)}{q(x)} + \log_D \frac{1}{p(x)} \right) + 1 \\ &= D_D(p \| q) + H_D(X) + 1. \end{aligned}$$

The lower bound follows from the same chain with the ceiling bounded below by its argument, i.e. by deleting the +1. $\blacksquare$

Your turn. In the chain above, which single algebraic step creates the divergence term? Writing $\frac{1}{q(x)} = \frac{p(x)}{q(x)} \cdot \frac{1}{p(x)}$ and splitting the logarithm. The first piece averages to $D_D(p \| q)$, the second to $H_D(X)$.

Your turn. Recover Proposition 4.3 from Proposition 6.2. Take $q = p$: then $D_D(p \| p) = 0$ and the bounds collapse to $H_D(X) \leq \mathbb{E}[|C(X)|] < H_D(X) + 1$.

Fano's code (1949)

Fano's construction is top-down: repeatedly split the alphabet into two groups of as nearly equal probability as possible, and let the split decide the next code digit.

Definition 6.3 (Fano code, binary case). Let p be a pmf on $\mathcal{X} = \{x_1, \dots, x_m\}$ and $X \sim p$.

1. Sort the symbols so that $p_1 \geq p_2 \geq \dots \geq p_m$.
2. Find the index r minimizing

$$\left| \sum_{i \leq r} p_i - \sum_{i > r} p_i \right|,$$

and split $\mathcal{X}$ into the two groups $\mathcal{X}_0 = \{x_i : i \leq r\}$ and $\mathcal{X}_1 = \{x_i : i > r\}$.

3. Define the first digit of every codeword in $\mathcal{X}_0$ to be 0, and in $\mathcal{X}_1$ to be 1.
4. Repeat steps (2) and (3) inside each group, appending one digit per round, until every group is a singleton.

This produces a code $C_F : \mathcal{X} \rightarrow \{0, 1\}^*$.

Because each symbol corresponds to a leaf of the splitting tree, C_F is a prefix code, and one can show

$$\mathbb{E}[|C_F(X)|] \leq H_D(X) + \underline{2}.$$

The bound is one symbol weaker than Shannon's, and indeed *Fano's code is not optimal in general*: balancing probabilities greedily at each split can be beaten by Huffman's bottom-up construction.

Your turn. Rank the three codes of this lecture — Shannon, Fano, optimal (Huffman) — by the guarantee each carries on $\mathbb{E}[|C(X)|]$, and say which of them is actually optimal. All three are bounded below by $H_D(X)$. Shannon's code guarantees $< H_D(X) + 1$, Fano's only $\leq H_D(X) + 2$, and the optimal code is at least as good as both, so it also satisfies $< H_D(X) + 1$. Neither Shannon's nor Fano's code is optimal in general; the optimum is attained by the Huffman code.

? Which quantity measures the loss from designing a code for q when the source is p ?